

TỔNG HỢP KIẾN THỨC CỐT LÕI

Học phần: SWE30003 – Kiến trúc và Thiết kế Phần mềm (Tuần 1 đến Tuần 7)

Tài liệu ôn tập hệ thống hóa

Ngôn ngữ: Tiếng Việt

Tuần 1: Khái niệm Cơ bản & Các vấn đề trong Thiết kế Phần mềm (Software Design Issues)

Thiết kế phần mềm là một quá trình sáng tạo nhằm chuyển hóa các yêu cầu bài toán (Problem) thành một giải pháp kiến trúc và kỹ thuật cụ thể (Solution). Bản thân bản mô tả giải pháp này cũng được gọi là một thiết kế (Design).

1.1 Phân biệt Bản chất Verification và Validation (V&V) trong SDLC

Đây là hai hoạt động kiểm định sống còn xuyên suốt vòng đời phát triển phần mềm (SDLC):

- Verification (Kiểm chứng - "Are we building the product right?"):** Đảm bảo rằng phần mềm được xây dựng tuân thủ chính xác theo các thông số kỹ thuật, tài liệu thiết kế và tiêu chuẩn đã đặt ra ở bước trước. Bản chất là kiểm tra tính đúng đắn của quá trình chuyển đổi giữa các pha (ví dụ: code có đúng thiết kế không, thiết kế có đúng SRS không). Hoạt động này diễn ra ở tất cả các pha của SDLC thông qua review, walkthrough, và tĩnh học.
- Validation (Xác thực - "Are we building the right product?"):** Đảm bảo rằng sản phẩm cuối cùng thực sự đáp ứng đúng nhu cầu, mong muốn thực tế và giải quyết được bài toán gốc của khách hàng/người dùng cuối. Bản chất là đối chiếu sản phẩm với thế giới thực. Hoạt động này tập trung mạnh ở pha Elicitation (nghiệm thu mockup/prototype) và Testing (Acceptance Testing, Beta Testing).

1.2 Thách thức trong Quản lý Độ phức tạp (Complexity Management)

Không thể loại bỏ hoàn toàn *độ phức tạp nội tại* (inherent/essential complexity) của một hệ thống quy mô lớn, nhưng mục tiêu tối thượng của thiết kế và kiến trúc là áp dụng các kỹ thuật phân rã (decomposition) và mô hình hóa phù hợp để kiểm soát và làm chủ nó, tránh phát sinh các khoản nợ kỹ thuật (technical debt).

Tuần 2: Tương tác Người dùng, Yêu cầu Chức năng & Thang đo Goal-Design

2.1 Thang đo Mục tiêu - Thiết kế (Goal-Design Scale)

Khi thu thập và đặc tả yêu cầu, các phát biểu của khách hàng nằm trên các mức độ chi tiết và mục đích khác nhau được phân loại theo thang đo của Søren Lauesen:

1. **Mức 1: Goal (Mục tiêu kinh doanh):** Lý do hệ thống tồn tại (ví dụ: "Giảm thời gian lập lịch trực từ 3 ngày xuống 2 giờ").
2. **Mức 2: Task (Nhiệm vụ người dùng):** Những việc người dùng cần làm để đạt mục tiêu, không phụ thuộc vào giao diện phần mềm (ví dụ: "Lập ca trực cho nhân viên y tế").
3. **Mức 3: Product Function (Chức năng sản phẩm):** Những gì hệ thống cung cấp để hỗ trợ nhiệm vụ (ví dụ: "Hệ thống tự động tính toán chi phí tài chính của bảng trực").
4. **Mức 4: Design (Thiết kế chi tiết):** Đặc tả giao diện, luồng màn hình hoặc cấu trúc dữ liệu cụ thể (ví dụ: "Nút bấm màu xanh, kích thước 20x40px, khi nhấn mở pop-up chọn nhân viên").

Nguyên tắc thiết kế sạch: Tránh sa đà vào Mức 4 quá sớm khi chưa làm rõ Mức 2 và Mức 3, nhằm giữ cho thiết kế có sự linh hoạt tối đa khi thay đổi giải pháp công nghệ.

2.2 So sánh Task Descriptions và Use Case Diagrams

Tiêu chí so sánh	Mô tả Nhiệm vụ (Task Descriptions)	Sơ đồ Ca sử dụng (Use Case Diagrams)
Bản chất	Tập trung vào chuỗi hành động thực tế của người dùng ngoài đời thực để hoàn thành công việc. Gồm các bước của con người kết hợp với sự hỗ trợ của máy tính.	Tập trung vào ranh giới hệ thống, phân rã các chức năng phần mềm mà tác nhân (Actor) có thể kích hoạt trực tiếp.
Ưu điểm	Rất gần gũi với quy trình nghiệp vụ thực tế; dễ dàng phát hiện các khoảng trống hỗ trợ; khách hàng không chuyên kỹ thuật rất dễ hiểu và xác thực.	Chuẩn hóa theo UML; trực quan hóa mối quan hệ phân rã chức năng (<<include>> , <<extend>>); tốt cho lập trình viên thiết kế test case kỹ thuật.
Ngữ cảnh phù hợp	Cực kỳ hiệu quả trong giai đoạn Elicitation khi nghiệp vụ phức tạp, mơ hồ và cần thiết kế giải pháp hỗ trợ tối ưu (Tasks & Support style).	Phù hợp khi hệ thống đã rõ ràng về mặt biên giới chức năng, cần bàn giao cho đội phát triển cấu trúc hóa hệ thống.

2.3 Mô hình hóa Miền dữ liệu (Domain Model)

Là bước cầu nối quan trọng, sử dụng sơ đồ lớp thực thể (High-level Entity-Relationship hoặc Domain Class Diagram) kèm giải thích tường tận để định hình cấu trúc dữ liệu cốt lõi, đảm bảo mọi thành viên có chung một ngôn ngữ nhất quán về mặt thuật ngữ nghiệp vụ (Ubiquitous Language).

Tuần 3: Thuộc tính Chất lượng & Kiểm định Tài liệu Đặc tả (SRS V&V)

3.1 Các thuộc tính chất lượng cốt lõi (Quality Attributes / Non-Functional Requirements)

Hệ thống không chỉ cần chạy đúng chức năng (Functional), mà phải chạy "tốt" theo các tiêu chí kiến trúc nghiêm ngặt:

- **Throughput (Băng thông/Năng suất):** Lượng công việc hoàn thành trong một đơn vị thời gian. Cần phân biệt rõ ràng giữa *Average Throughput* (băng thông trung bình) và *Peak Throughput* (băng thông tại thời điểm đỉnh tải) để cấu hình hạ tầng chính xác, tránh nghẽn cổ chai.
- **Scalability (Khả năng mở rộng):** Gồm *Scaling Up (Vertical)* - tăng cường tài nguyên phần cứng (CPU, RAM) trên một máy chủ đơn lẻ (có giới hạn vật lý và chi phí tăng tiến lũy thừa); và *Scaling Out (Horizontal)* - bổ sung thêm nhiều thực thể máy chủ giá rẻ vào cụm để chia sẻ tải (yêu cầu kiến trúc phần mềm không lưu trạng thái - stateless).
- **Security (An ninh):** Tách biệt rõ ràng giữa **Authentication** (Xác thực danh tính - Bạn là ai? qua Password, JWT, MFA) và **Authorization** (Phân quyền - Bạn được phép làm gì? thông qua RBAC, ABAC).
- **Testability (Khả năng kiểm thử):** Mức độ dễ dàng để thiết lập các kịch bản thực thi nhằm phát hiện lỗi. Được biểu diễn thông qua việc module hóa, cung cấp các điểm quan sát dữ liệu đầu ra và các điểm kiểm soát đầu vào rõ ràng.

Nguyên lý bất biến: Các thuộc tính chất lượng không bao giờ tồn tại độc lập mà luôn có sự xung đột và ràng buộc lẫn nhau (Trade-offs). Ví dụ: Việc tăng cường Security (mã hóa dữ liệu, xác thực nhiều bước) chắc chắn sẽ làm giảm Performance/Throughput của hệ thống. Hoặc việc tăng cường Scalability (chia tách microservices) sẽ làm giảm Testability và tăng độ phức tạp vận hành.

3.2 Kịch bản Thuộc tính Chất lượng (Quality Attribute Scenarios)

Để một yêu cầu phi chức năng có thể kiểm chứng được (Verifiable), nó phải được viết dưới dạng một kịch bản chuẩn gồm 6 thành phần: *Source of Stimulus* (Nguồn kích thích), *Stimulus* (Kích thích), *Artifact* (Thành phần hệ thống chịu tác động), *Environment* (Ngữ cảnh môi trường), *Response* (Phản ứng của hệ thống), và *Response Measure* (Thước đo định lượng phản ứng - ví dụ: trong vòng dưới 2 giây, sai số dưới 1%).

3.3 Kiểm định cấu trúc tài liệu SRS (Structure Check)

Đảm bảo tài liệu đặc tả yêu cầu phần mềm (SRS) không chứa các từ ngữ mơ hồ, mọi yêu cầu phải có mục đích rõ ràng, có giải thích cho các sơ đồ đồ họa và đặc biệt là thông tin không được dư thừa (sử dụng cross-reference thay vì copy-paste) nhằm tránh mâu thuẫn khi cập nhật.

Tuần 4: Vai trò của Trừu tượng hóa (Abstraction) trong Kiến trúc Phần mềm

4.1 Bản chất của Trừu tượng hóa (Abstraction)

Trừu tượng hóa là công cụ tư duy tối thượng giúp lập trình viên và kiến trúc sư làm chủ sự phức tạp bằng cách tập trung hoàn toàn vào các đặc điểm quan trọng cốt lõi của một thực thể tại một thời điểm, đồng thời che giấu đi những chi tiết triển khai chi tiết chưa cần thiết ở tầng đó. Trừu tượng hóa chính là nền móng cốt lõi để thực hiện phân rã module hệ thống.

4.2 Xây dựng Ngôn ngữ và Vốn từ vựng làm việc (Working Vocabulary)

Một kỹ sư phần mềm xuất sắc không đơn thuần là viết các dòng mã thực thi, họ là người *xây dựng một vốn từ vựng làm việc chặt chẽ* cho hệ thống. Thông qua việc định nghĩa các lớp trừu tượng, interfaces, các kiểu dữ liệu đặc thù của miền nghiệp vụ và áp dụng các mẫu thiết kế (Design Patterns), họ tạo ra một hệ ngôn ngữ có tính đóng gói cao, giúp mã nguồn đọc như một bài văn xuôi mạch lạc, có thể dễ dàng mở rộng và giảm thiểu tối đa nợ kỹ thuật.

Tuần 5: Thiết kế Hướng Đối tượng I (Object Design - Responsibility-Driven Design)

5.1 Triết lý Responsibility-Driven Design (RDD)

Ứng dụng phần mềm hướng đối tượng thực chất là một tập hợp các đối tượng tương tác, cộng tác với nhau. Thiết kế theo RDD coi mỗi đối tượng như một sinh thể sống có các đặc tính sau:

- **Role (Vai trò):** Một tập hợp các trách nhiệm có liên quan chặt chẽ với nhau mà đối tượng đảm nhận trong một ngữ cảnh cụ thể.
- **Responsibility (Trách nhiệm):** Nghĩa vụ của một đối tượng. Gồm hai loại: *Trách nhiệm hành động (Doing)* - thực hiện một tác vụ toán học/nghiệp vụ; và *Trách nhiệm ghi nhớ (Knowing)* - lưu trữ dữ liệu, trạng thái hoặc biết đối tượng nào khác để hỏi thông tin.
- **Collaboration (Cộng tác):** Sự tương tác qua lại giữa các đối tượng để hoàn thành một trách nhiệm lớn hơn vượt quá năng lực của một đối tượng đơn lẻ.

5.2 Nguyên tắc phân tách Giao diện và Triển khai (Interface vs. Implementation)

Giao diện (Interface) định nghĩa "đối tượng làm được việc gì" (khế ước cam kết), còn Implementation định nghĩa "đối tượng làm việc đó như thế nào". Việc lập trình phụ thuộc vào Interface giúp lỏng lẻo sự liên kết (Loose Coupling), cho phép thay đổi phần core triển khai bên trong hoặc chuyển đổi nền tảng (portability) mà không làm ảnh hưởng đến mã nguồn của các module gọi nó.

Nguyên tắc bài xích "Data Holder Class": Tránh thiết kế các lớp chỉ thuần túy chứa dữ liệu (chỉ có getter/setter nông cạn) mà không có hành vi tự quản lý dữ liệu đó. Điều này vi phạm tính đóng gói và biến các lớp khác thành "God Class" xử lý hệ dữ liệu.

Tuần 6: Thiết kế Hướng Đối tượng II (Object Design - Design Refinement)

Tập trung vào các quy tắc tinh chỉnh hệ thống lớp để đảm bảo tính tái sử dụng, dễ bảo trì thông qua các quy tắc heuristic thực tế từ Arthur Riel và Bertrand Meyer.

6.1 Thiết lập Phân cấp Lớp kế thừa chuẩn mực (Class Hierarchies)

- Sử dụng **Abstract Classes** để cải thiện khả năng tái sử dụng mã nguồn dùng chung và đảm bảo tính di động (portability) của hệ thống.
- Vapor Class (Lớp hơi/Lớp giả):** Là các lớp trừu tượng nằm ở tầng rất cao trong sơ đồ phân cấp cấu trúc nhằm định hình khái niệm tư duy tổng quát hơn là thực thi kỹ thuật. Chúng cực kỳ hữu ích cho việc định hướng kiến trúc mở rộng lâu dài cho hệ thống.
- Tác hại của God Class (Lớp vạn năng):** Một lớp nắm giữ quá nhiều trách nhiệm, kiểm soát hầu hết dòng chảy dữ liệu của các lớp khác xung quanh. Nó phá vỡ hoàn toàn nguyên lý phân rã cấu trúc, tăng độ kết dính mãnh liệt (Tight Coupling) và biến hệ thống hướng đối tượng trở lại thành lập trình thủ tục biến tướng, cực kỳ khó bảo trì.

6.2 Bài toán Đa kế thừa (Multiple Inheritance) & Case Study Trận cờ vua

Tình huống: Khi thiết kế ứng dụng Cờ vua, ta có nên mô hình hóa quân Hậu (Queen) kế thừa từ cả quân Xe (Rook) và quân Tịch (Bishop) để tái sử dụng luật di chuyển (thẳng và chéo) hay không?

Giải pháp & Heuristics áp dụng: Không nên áp dụng đa kế thừa trực tiếp từ hai lớp cụ thể trong trường hợp này. Quy tắc heuristic chỉ ra rằng mối quan hệ kế thừa phải tuân thủ nghiêm ngặt quan hệ bản chất loại hình "IS-A". Một quân Hậu không phải đồng thời vừa là quân Xe vừa là quân Tịch về mặt bản chất danh tính sinh học trong trò chơi. Việc cố tình đa kế thừa sẽ dẫn đến xung đột mã nguồn (bài toán kim cương - Diamond Problem) và ghép cặp chặt một cách sai lệch. Cách tiếp cận chuẩn mực hơn là sử dụng **Delegation (Ủy thác hành vi)** hoặc tách biệt cơ chế tính toán nước đi hợp lệ thành các chiến lược độc lập (áp dụng Strategy Pattern) và inject vào lớp quân cờ.

Tuần 7: Các Mẫu Thiết kế Hệ thống Kiến trúc (Design Patterns & Idioms)

Mẫu thiết kế (Design Pattern) là các giải pháp đã được chuẩn hóa để giải quyết các vấn đề thiết kế phổ biến xuất hiện lặp đi lặp lại trong công nghiệp phần mềm.

7.1 Idiom: Delegation (Ủy thác)

Là một kỹ thuật thiết kế mạnh mẽ thay thế cho kế thừa phần cứng. Thay vì một đối tượng tự mình thực thi hoặc kế thừa hành vi từ lớp cha, nó giữ một tham chiếu đến một đối tượng trợ giúp khác và chuyển tiếp (forward) yêu cầu thực thi tác vụ cho đối tượng đó đảm nhiệm. Giúp chuyển đổi linh hoạt hành vi ngay trong thời gian chạy (Runtime).

7.2 Hệ thống 8 Design Patterns Cơ bản & Bản chất Kiến trúc

Tên Pattern	Bản chất Ý tưởng Thiết kế	Cấu trúc thành phần & Mối tương tác cốt lõi
Template Method	Định nghĩa bộ khung (bộ xương) của một thuật toán trong một phương thức của lớp cha, nhưng trì hoãn việc triển khai một số bước cụ thể xuống các lớp con mà không làm thay đổi cấu trúc tổng thể của thuật toán.	Lớp cha abstract chứa phương thức template gọi các phương thức trừu tượng nhỏ (<code>primitiveOperations()</code>). Các lớp con cụ thể ghi đè (override) các bước nhỏ này để tùy biến chi tiết.
Factory Method	Định nghĩa một giao diện để khởi tạo một đối tượng, nhưng để các lớp con tự quyết định lớp nào sẽ được khởi tạo. Khởi tạo đối tượng mà không để lộ logic khởi tạo ra ngoài.	Lớp <code>Creator</code> khai báo phương thức factory trả về một đối tượng kiểu <code>Product</code> . Các <code>ConcreteCreator</code> sẽ override phương thức này để trả về instance của <code>ConcreteProduct</code> cụ thể.
Strategy	Định nghĩa một họ các thuật toán giống nhau, đóng gói từng thuật toán lại thành các lớp độc lập và giúp chúng có thể hoán đổi linh hoạt cho nhau ngay trong runtime tùy theo ngữ cảnh của ứng dụng.	Lớp <code>Context</code> giữ một tham chiếu đến interface <code>Strategy</code> . Khi cần thực thi, Context gọi phương thức thông qua interface đó. Các lớp <code>ConcreteStrategy</code> cụ thể hiện thực thuật toán riêng biệt.
Composite	Cho phép tổ chức các đối tượng theo cấu trúc dạng cây để biểu diễn các mối quan hệ phân cấp từ một phần đến toàn bộ (part-whole hierarchies). Giúp client xử lý các đối tượng đơn lẻ và tập hợp các đối tượng một cách đồng nhất.	Thành phần <code>Component</code> định nghĩa giao diện chung. Lớp <code>Leaf</code> đại diện cho phần tử lá không có con. Lớp <code>Composite</code> chứa một danh sách các <code>Component</code> con và chuyển tiếp yêu cầu đến chúng.
Observer	Định nghĩa một mối quan hệ phụ thuộc một-nhiều giữa các đối tượng sao cho khi một đối tượng thay đổi trạng thái, tất cả các đối tượng phụ thuộc của nó (Observers) sẽ tự động nhận được thông báo và cập nhật.	Đối tượng <code>Subject</code> duy trì danh sách các <code>Observer</code> , cung cấp các hàm <code>attach()</code> , <code>detach()</code> , và <code>notify()</code> . Khi có biến động, Subject duyệt danh sách và gọi hàm <code>update()</code> trên từng Observer.
Null Object	Tránh việc liên tục kiểm tra giá trị <code>null</code> gây rác mã nguồn bằng cách cung cấp một đối tượng rỗng kế thừa từ interface chung, đối tượng này hiện thực các phương thức với hành vi mặc định (không làm gì cả).	Một interface/abstract class chung, các lớp con thực thi thật và một lớp <code>NullObject</code> hiện thực các phương thức trống an toàn. Giúp mã nguồn chạy liên tục không lo lỗi crash <code>NullPointerException</code> .
Singleton		

Tên Pattern	Bản chất Ý tưởng Thiết kế	Cấu trúc thành phần & Mối tương tác cốt lõi
	Đảm bảo rằng một lớp duy nhất chỉ có thể có tối đa một instance (thực thể) duy nhất được khởi tạo trong suốt vòng đời của ứng dụng và cung cấp một điểm truy cập toàn cục duy nhất đến instance đó.	Lớp có hàm khởi tạo private (<code>private Constructor</code>) để ngăn chặn từ khóa <code>new</code> từ bên ngoài, đồng thời sở hữu một biến tĩnh lưu chính nó và một phương thức tĩnh <code>getInstance()</code> trả về thực thể duy nhất đó.
Command	Đóng gói một yêu cầu tác vụ thành một đối tượng độc lập, từ đó cho phép tham số hóa các client với các yêu cầu khác nhau, xếp hàng yêu cầu, ghi log tác vụ và hỗ trợ khả năng hủy bỏ hành vi (Undo/Redo).	Interface <code>Command</code> khai báo hàm <code>execute()</code> . Lớp <code>ConcreteCommand</code> ràng buộc hành vi giữa một thực thể nhận lệnh (<code>Receiver</code>) và một hành động. <code>Invoker</code> chịu trách nhiệm kích hoạt thực thi lệnh.